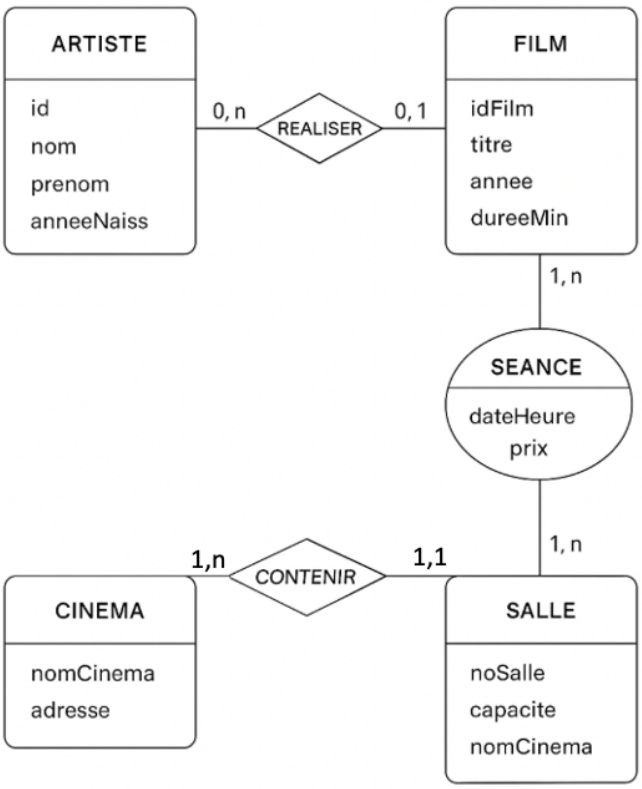
ECE 2025/2026
Module : Base de Données
Enseignante : Salma Sassi

Correction du TD/TP3 : Manipulation d'une base de données SQL (DDL & DML)

1. MCD



2. MLD (schéma relationnel)

ARTISTE(**id**, nom, prenom, anneeNaiss)
FILM (**idFilm**, titre, annee, dureeMin, #id)
CINEMA (**nomCinema**, adresse)
SALLE (**noSalle**, capacite, #nomCinema)
SEANCE (#idFilm, #noSalle, dateHeure, prix)

Script de création – Base « Cinéma » (DDL)

```
-- ==========================================

-- 1) Création de la base

-- ==========================================

CREATE TABLE Artiste (

    id INT PRIMARY KEY,

    nom VARCHAR(50) NOT NULL,

    prenom VARCHAR(50) NOT NULL,

    anneeNaiss INT

);

CREATE TABLE Film (

    idFilm INT PRIMARY KEY,

    titre VARCHAR(100) NOT NULL,

    annee INT NOT NULL,

    dureeMin INT NOT NULL,

    idRealisateur INT,

    FOREIGN KEY (idRealisateur) REFERENCES Artiste(id)

);

CREATE TABLE Cinema (

    nomCinema VARCHAR(50) PRIMARY KEY,

    adresse VARCHAR(100) NOT NULL
```

```
);

CREATE TABLE Salle (
    noSalle INT,
    nomCinema VARCHAR(50),
    capacite INT NOT NULL,
    PRIMARY KEY (noSalle, nomCinema),
    FOREIGN KEY (nomCinema) REFERENCES Cinema(nomCinema)
);

CREATE TABLE Seance (
    idFilm INT,
    noSalle INT,
    nomCinema VARCHAR(50),
    dateHeure DATETIME,
    prix DECIMAL(5,2) NOT NULL,
    PRIMARY KEY (idFilm, noSalle, nomCinema, dateHeure),
    FOREIGN KEY (idFilm) REFERENCES Film(idFilm),
    FOREIGN KEY (noSalle, nomCinema) REFERENCES Salle(noSalle, nomCinema)
);

-- ============================================
-- 2) Insertions
-- ============================================

INSERT INTO Artiste (id, nom, prenom, anneeNaiss) VALUES
(1, 'Nolan', 'Christopher', 1970),
(2, 'Villeneuve', 'Denis', 1967),
```

```
(3, 'Coppola', 'Sofia', 1971);

INSERT INTO Film (idFilm, titre, annee, dureeMin, idRealisateur) VALUES
(101, 'Inception', 2010, 148, 1),
(102, 'Dune', 2021, 155, 2),
(103, 'Lost in Translation', 2003, 102, 3);

INSERT INTO Cinema (nomCinema, adresse) VALUES
('Lumiere', '12 rue des Arts, Lyon'),
('Ecrans', '5 avenue du Cinema, Paris');

INSERT INTO Salle (noSalle, nomCinema, capacite) VALUES
(1, 'Lumiere', 120),
(2, 'Lumiere', 80),
(1, 'Ecrans', 200);

INSERT INTO Seance (idFilm, noSalle, nomCinema, dateHeure, prix) VALUES
(101, 1, 'Lumiere', '2026-04-01 20:00:00', 10.50),
(101, 2, 'Lumiere', '2026-04-01 22:00:00', 10.50),
(102, 1, 'Ecrans', '2026-04-02 19:30:00', 12.00),
(103, 2, 'Lumiere', '2026-04-03 18:00:00', 9.00);

-- ==========================================
-- 3) Vérification
-- ==========================================

SELECT * FROM Artiste;
SELECT * FROM Film;
```

```
SELECT * FROM Cinema;

SELECT * FROM Salle;

SELECT * FROM Seance;
```

3) « Mise à jour / Suppression »

A) UPDATE

1) +10 % sur le prix des séances du film *Inception* (idFilm = 101)

```
UPDATE Seance
SET prix = ROUND(prix * 1.10, 2)
WHERE idFilm = 101;
```

2) Fixer le prix à 12.00 pour les séances dans "Écrans" après le 2025-10-12

```
UPDATE Seance
SET prix = 12.00
WHERE nomCinema = 'Écrans'
  AND dateHeure > '2025-10-12 00:00:00';
```

3) +15 minutes sur la durée de *Dune*

```
UPDATE Film
SET dureeMin = dureeMin + 15
WHERE titre = 'Dune';
```

4) Mettre `idRealisateur = NULL` pour les films réalisés par l'artiste id = 3

```
UPDATE Film
SET idRealisateur = NULL
WHERE idRealisateur = 3;
```

5) −5 % sur le prix des séances dont la salle a une capacité < 100

(jointure avec SALLE)

```
UPDATE Seance s
JOIN   Salle  sa
  ON   s.nomCinema = sa.nomCinema
 AND   s.noSalle   = sa.noSalle
SET s.prix = ROUND(s.prix * 0.95, 2)
WHERE sa.capacite < 100;
```

B) DELETE (référentiel d'intégrité)

6) Tenter de supprimer le film *Inception*

```
DELETE FROM Film WHERE idFilm = 101;  -- ÉCHOUERA
```

Pourquoi ça échoue ?
La contrainte `fk_seance_film ON DELETE RESTRICT` empêche de supprimer un film **tant qu'il existe des séances** associées.

Comment le faire proprement ? (deux options)

- **Option A** : supprimer d'abord les séances du film, puis le film :
- `DELETE FROM Seance WHERE idFilm = 101;`
- `DELETE FROM Film  WHERE idFilm = 101;`
- **Option B (conception différente)** : si, **par choix métier**, on accepte la suppression en cascade, il faudrait **modifier** la FK pour `ON DELETE CASCADE` (changement de modèle, **non demandé ici**).

7) Supprimer le cinéma "Lumière" – observer l'effet cascade

```
DELETE FROM Cinema WHERE nomCinema = 'Lumière';
```

Comportement attendu :

- `ON DELETE CASCADE` sur `Salle` ⇒ **toutes les salles** du cinéma *Lumière* sont supprimées automatiquement.
- Mais la table **Seance** référence `(nomCinema, noSalle)` avec `ON DELETE RESTRICT`. ⇒ S'il existe des **séances** dans ces salles, **la suppression du cinéma sera bloquée** (RESTRICT empêche d'orpheliner des séances).

Procédure correcte (sans changer le modèle) :

```
DELETE FROM Seance
WHERE nomCinema = 'Lumière';

DELETE FROM Cinema
WHERE nomCinema = 'Lumière';
```

➔ Résultat : les salles de *Lumière* seront supprimées en cascade par la FK de `Salle` → `Cinema`.

(Si l'on souhaitait tout automatiser, on mettrait aussi `ON DELETE CASCADE` sur `fk_seance_salle`, mais ce n'est pas le choix retenu ici.)

8) Supprimer toutes les séances avant le 2025-10-12

```
DELETE FROM Seance
WHERE dateHeure < '2025-10-12 00:00:00';
```

Conclusion

- Les UPDATE utilisent des conditions ciblées (voire des JOIN).
- Les DELETE sont contraints par les FK (`RESTRICT`, `CASCADE`, `SET NULL`).
- La cohérence est contrôlée par le modèle (les contraintes) — c'est la clé à faire passer aux étudiants.